

Web Engineering & Development (SWE 363)

Interactive Front-End Development

Announcements

- Start working on Project Milestone 3: Requirements
- Demo submission, commit each TODO

In today's lecture:

- Using JavaScript with HTML
- Document Object Model (DOM)
- Using third-party web APIs (JavaScript)
- JavaScript Object Notation (JSON)

Reference:

- Zybook: 5.1 to 5.4, 5.17

5.1 Using JavaScript with HTML

What is JavaScript?

JavaScript is a **programming language** that runs in web browsers

- Makes web pages **interactive** and **dynamic**
- Can change content, respond to user actions
- Works alongside HTML and CSS

Why JavaScript?

HTML = Structure (skeleton)

CSS = Styling (appearance)

JavaScript = Behavior (interactivity)

```
<!-- HTML: Just structure -->  
<button>Click Me</button>  
<p id="message">Hello</p>
```

```
// JavaScript: Makes it interactive  
// When button clicked → change message
```

Three Ways to Add JavaScript

1. Inline (inside HTML tags)

```
<button onclick="alert('Hello!')">Click Me</button>
```

2. Internal (in `<script>` tags)

```
<script>  
    function showMessage() { alert('Hello from JavaScript!'); }  
</script>
```

3. External (separate .js file) **Best Practice**

```
<script src="script.js"></script>
```

External JavaScript - The Right Way

HTML File (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Welcome</h1>
  <button id="myButton">Click Me</button>
</body>
</html>
```


External JavaScript - The Right Way

JavaScript File (script.js)

```
console.log("Page loaded!");
```

Key Points:

- HTML and JavaScript are in **separate files**
- Use `defer` attribute for proper loading order
- Clean separation of structure and behavior

The **defer** Attribute

```
<script src="script.js" defer></script>
```

What it does:

- Downloads JavaScript file while HTML continues loading
- Runs JavaScript **after** HTML is fully loaded
- Prevents errors when JavaScript tries to access HTML elements

Without **defer** : JavaScript might run before HTML elements exist!

When Does JavaScript Run?

Page Loading Order:

1. **HTML** loads and creates elements
2. **CSS** styles the page
3. **JavaScript** runs and makes it interactive

Safe Way to Wait for HTML:

```
document.addEventListener("DOMContentLoaded", function() {  
    // This code runs ONLY after HTML is ready  
    console.log("HTML is fully loaded!");  
});
```

Common Mistakes to Avoid

1. Forgetting `defer`

```
<!-- BAD: JavaScript runs before HTML -->  
<script src="script.js"></script>
```

```
<!-- GOOD: JavaScript waits for HTML -->  
<script src="script.js" defer></script>
```

Common Mistakes to Avoid

2. Not waiting for DOM

```
// BAD: Element might not exist yet
document.getElementById("myButton").addEventListener("click", ...);

// GOOD: Wait for DOM to be ready
document.addEventListener("DOMContentLoaded", function() {
    document.getElementById("myButton").addEventListener("click", ...);
});
```

5.2 Document Object Model (DOM)

What is the DOM?

The **DOM** is like a **family tree** of your HTML page

- Every HTML element becomes a **node** in the tree
- JavaScript can **find** and **change** these nodes
- It's the bridge between HTML and JavaScript

DOM Tree Structure

```
<html>
  <head> <title>My Page</title> </head>
  <body>
    <h1>Welcome</h1>
    <p id="message">Hello World</p>
    <button>Click Me</button>
  </body>
</html>
```

```
html
├── head
│   └── title ("My Page")
└── body
    ├── h1 ("Welcome")
    ├── p#message ("Hello World")
    └── button ("Click Me")
```


Why Do We Need the DOM?

Problem: JavaScript needs to talk to HTML elements

Solution: The DOM gives JavaScript a way to:

- **Find** HTML elements
- **Read** their content
- **Change** their content
- **Add** new elements
- **Remove** elements

Finding Elements - The Basics

By ID (Most Common)

```
// Find element with id="myButton"  
const button = document.getElementById("myButton");
```

By CSS Selector

```
// Find first element with class="highlight"  
const element = document.querySelector(".highlight");  
  
// Find first <p> tag  
const paragraph = document.querySelector("p");
```

Finding Elements - More Options

By Class Name

```
// Find ALL elements with class="item"  
const items = document.getElementsByClassName("item");
```

By Tag Name

```
// Find ALL <p> tags  
const paragraphs = document.getElementsByTagName("p");
```

Multiple Elements with Query Selector

```
// Find ALL elements with class="highlight"  
const highlights = document.querySelectorAll(".highlight");
```

Changing Element Content

Text Content (Safe)

```
const element = document.getElementById("message");  
element.textContent = "New text here";
```

HTML Content (Use Carefully)

```
const element = document.getElementById("message");  
element.innerHTML = "<strong>Bold text</strong>";
```

Rule: Use `textContent` for plain text, `innerHTML` only when you need HTML

Changing Element Properties

Style Properties

```
const element = document.getElementById("myDiv");  
element.style.color = "red";  
element.style.backgroundColor = "blue";  
element.style.fontSize = "20px";
```

Other Properties

```
const link = document.getElementById("myLink");  
link.href = "https://www.google.com";  
link.target = "_blank";
```

Adding and Removing Elements

Create New Element

```
const newParagraph = document.createElement("p");  
newParagraph.textContent = "This is a new paragraph";
```

Add to Page

```
const container = document.getElementById("container");  
container.appendChild(newParagraph);
```

Remove Element

```
const element = document.getElementById("toRemove");  
element.remove();
```

Fun Examples

Repository:

<https://github.com/bradtraversy/50projects50days?tab=readme-ov-file>

Common DOM Methods Cheat Sheet

Finding Elements

- `getElementById("id")` - Find by ID
- `querySelector(".class")` - Find first match
- `querySelectorAll(".class")` - Find all matches

Changing Content

- `textContent = "text"` - Set text content
- `innerHTML = "<p>HTML</p>"` - Set HTML content

Common DOM Methods Cheat Sheet

Changing Style

- `style.property = "value"` - Change CSS property

Creating Elements

- `createElement("tag")` - Create new element
- `appendChild(element)` - Add to page
- `remove()` - Remove element

Best Practices

1. Always Wait for DOM

```
document.addEventListener("DOMContentLoaded", function() { });
```

2. Use textContent for Text

```
element.textContent = "Safe text"; // Good
```

```
element.innerHTML = "<strong>Bold</strong>"; // Only when you need HTML
```

3. Check if Element Exists

```
const element = document.getElementById("myElement");  
if (element) { element.textContent = "Found it!"; }
```

5.3 Using Third-Party Web APIs

What is a Web API?

API = Application Programming Interface

- A way for your website to **talk to other websites**
- Get data from external services
- Make your website more powerful and dynamic

Examples:

Weather data, News articles, Stock prices, Maps, Social media posts, etc.

How Do Web APIs Work?

Your Website → Send Request → API Server → Send Data Back → Display Data

Real Example:

1. User clicks "Get Weather"
2. Your site asks weather API for Dammam weather
3. API sends back temperature, humidity, etc.
4. Your site shows the weather to user

The `fetch()` Function

`fetch()` is how JavaScript talks to APIs

```
// Basic fetch request
fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("Something went wrong!");
  });
```

What happens:

Send request to API → Wait for response → Convert to JavaScript object → Use the data

Understanding Promises

`fetch()` returns a **Promise** - it's like ordering food:

```
// Order food (start request)
fetch("https://api.example.com/data")
  .then(function(response) {
    // Food arrives (response received)
    return response.json(); // Unwrap the food
  })
  .then(function(data) {
    // Eat the food (use the data)
    console.log(data);
  })
  .catch(function(error) {
    // Something went wrong
    console.log("Order failed!");
  });
```

Modern Way: async/await

Easier to read than `.then()` chains:

```
async function getData() {  
  try {  
    // Wait for the response  
    const response = await fetch("https://api.example.com/data");  
  
    // Wait for JSON conversion  
    const data = await response.json();  
  
    // Use the data  
    console.log(data);  
  } catch (error) {  
    console.log("Something went wrong!");  
  }  
}
```


Fun Examples

Repository:

<https://github.com/bradtraversy/50projects50days?tab=readme-ov-file>

API URLs and Parameters

Many APIs need information in the URL:

```
// Base URL
const baseUrl = "https://api.openweathermap.org/data/2.5/weather";

// Parameters
const city = "Dammam";
const apiKey = "your_api_key_here";
const units = "metric";

// Build complete URL
const url = `${baseUrl}?q=${city}&appid=${apiKey}&units=${units}`;

// Result: https://api.openweathermap.org/data/2.5/weather?q=Dammam&appid=your_api_key_here&units=metric

//city name to coordinates:
// https://nominatim.openstreetmap.org/search?q=${cityName}&format=json&limit=1
//weather data:
// https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}&current_weather=true
```

Handling Different Response Types

Check if Request was Successful

```
const response = await fetch("https://api.example.com/data");

if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${response.statusText}`);
}

const data = await response.json();
```

Common HTTP Status Codes

200: Success **404**: Not Found **401**: Unauthorized (bad API key) **500**: Server Error

Popular Free APIs to Try

API Name	URL
OpenWeatherMap	<code>https://api.openweathermap.org/data/2.5/weather</code>
Quotable	<code>https://api.quotable.io/random</code>
Cat Facts	<code>https://catfact.ninja/fact</code>
Random User	<code>https://randomuser.me/api/</code>

5.4 JavaScript Object Notation (JSON)

What is JSON?

JSON = JavaScript Object Notation

- A way to **store and share data**
- Easy for humans to read
- Easy for computers to understand
- Used by most web APIs

Think of it as: A universal language for data

Why Do We Need JSON?

Problem: Different programming languages need to share data

Solution: JSON works everywhere!

- JavaScript ✓
- Python ✓
- Java ✓
- C# ✓
- PHP ✓
- And many more!

JSON vs JavaScript Objects

JavaScript Object:

```
{ name: "Ahmed",  
  age: 25,  
  isStudent: true,  
  courses: ["Math", "Physics"] }
```

JSON:

```
{ "name": "Ahmed",  
  "age": 25,  
  "isStudent": true,  
  "courses": ["Math", "Physics"] }
```

Key Difference: JSON requires **quotes around property names**

JSON Rules

1. Property Names Must Be in Quotes

```
{  
  "name": "Ahmed",      Correct  
  "age": 25,            Correct  
  name: "Ahmed"         Wrong – no quotes  
}
```

2. Only These Data Types Allowed

Strings: "Hello"

Numbers: 42 or 3.14

Booleans: true or false

null: null

Arrays: [1, 2, 3]

Objects: {"key": "value"}

Converting JSON to JavaScript

From JSON String to JavaScript Object

```
// JSON string (from API)
const jsonString = '{"name": "Ahmed", "age": 25}';

// Convert to JavaScript object
const person = JSON.parse(jsonString);

// Now you can use it
console.log(person.name); // "Ahmed"
console.log(person.age);  // 25
```

Converting JSON to JavaScript

From JavaScript Object to JSON String

```
// JavaScript object
const person = {
  name: "Ahmed",
  age: 25,
  isStudent: true
};

// Convert to JSON string
const jsonString = JSON.stringify(person);
console.log(jsonString); // '{"name":"Ahmed","age":25,"isStudent":true}'
```

Safe Property Access

The Problem

```
// What if the API doesn't return expected data?  
const temperature = weatherData.main.temp; // Error if main is undefined!
```

The Solution: Optional Chaining

```
// Safe access – won't crash if property doesn't exist  
const temperature = weatherData?.main?.temp;  
const humidity = weatherData?.main?.humidity;  
  
// With default values  
const temperature = weatherData?.main?.temp ?? "Unknown";  
const humidity = weatherData?.main?.humidity ?? 0;
```

Working with Arrays in JSON

JSON with Array

```
{ "students": [ {"name": "Ahmed", "grade": 95}, {"name": "Sara", "grade": 88} ] }
```

JavaScript Code

```
const data = JSON.parse(jsonString);

// Loop through the array
data.students.forEach(student => {
  console.log(`${student.name}: ${student.grade}`);
});
```

Common JSON Mistakes

1. Missing Quotes Around Property Names

```
// WRONG
{
  name: "Ahmed",
  age: 25
}

// CORRECT
{
  "name": "Ahmed",
  "age": 25
}
```

Common JSON Mistakes

2. Using Single Quotes

```
// WRONG
{
  'name': 'Ahmed'
}

// CORRECT
{
  "name": "Ahmed"
}
```

Common JSON Mistakes

3. Trailing Commas

```
// WRONG
{
  "name": "Ahmed",
  "age": 25,
}
```

```
// CORRECT
{
  "name": "Ahmed",
  "age": 25
}
```


JSON Validation Tools

Online Validators

- **JSONLint:** <https://jsonlint.com/>
- **JSON Formatter:** <https://jsonformatter.curiousconcept.com/>

Browser Console

```
// Test if JSON is valid
try {
  const obj = JSON.parse('{ "name": "Ahmed" }');
  console.log("Valid JSON!");
} catch (error) {
  console.log("Invalid JSON:", error.message);
}
```

30m

Demo

5.1 DOM Manipulation and API

API Key: 9c29da573838fd8cdd561179419142d7

API Key: d51f2f00c3b137ccfd135bd8f9dd50aa

Next Class

- React